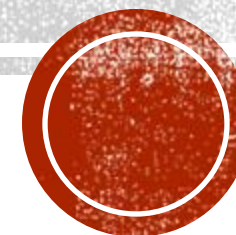


APRENDIZAJE SUPERVISADO



Anibal Sosa, PhD



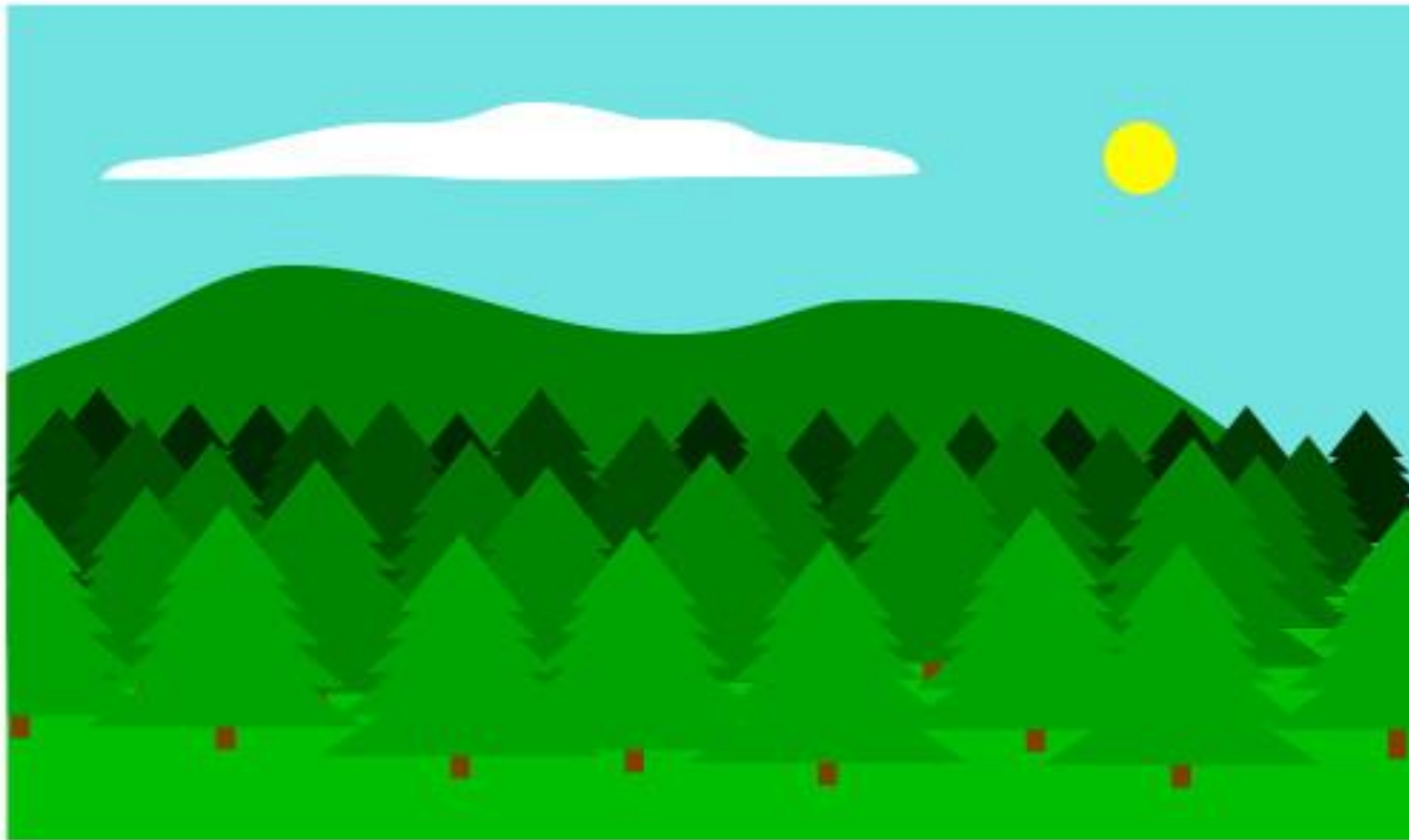
OBJETIVOS DE APRENDIZAJE

- **[OT1]** Identificar problemáticas que se puedan resolver a partir de modelos de aprendizaje supervisado, ya sea de clasificación o de regresión.
- **[OT2]** Aplicar modelos de aprendizaje supervisado a conjuntos de datos para responder a preguntas de negocio involucrando modelos de aprendizaje supervisado, utilizando el lenguaje Python.
- **[OT5]** Establecer los mejores valores de los parámetros para modelos de aprendizaje supervisado





MODELOS DE ENSAMBLE

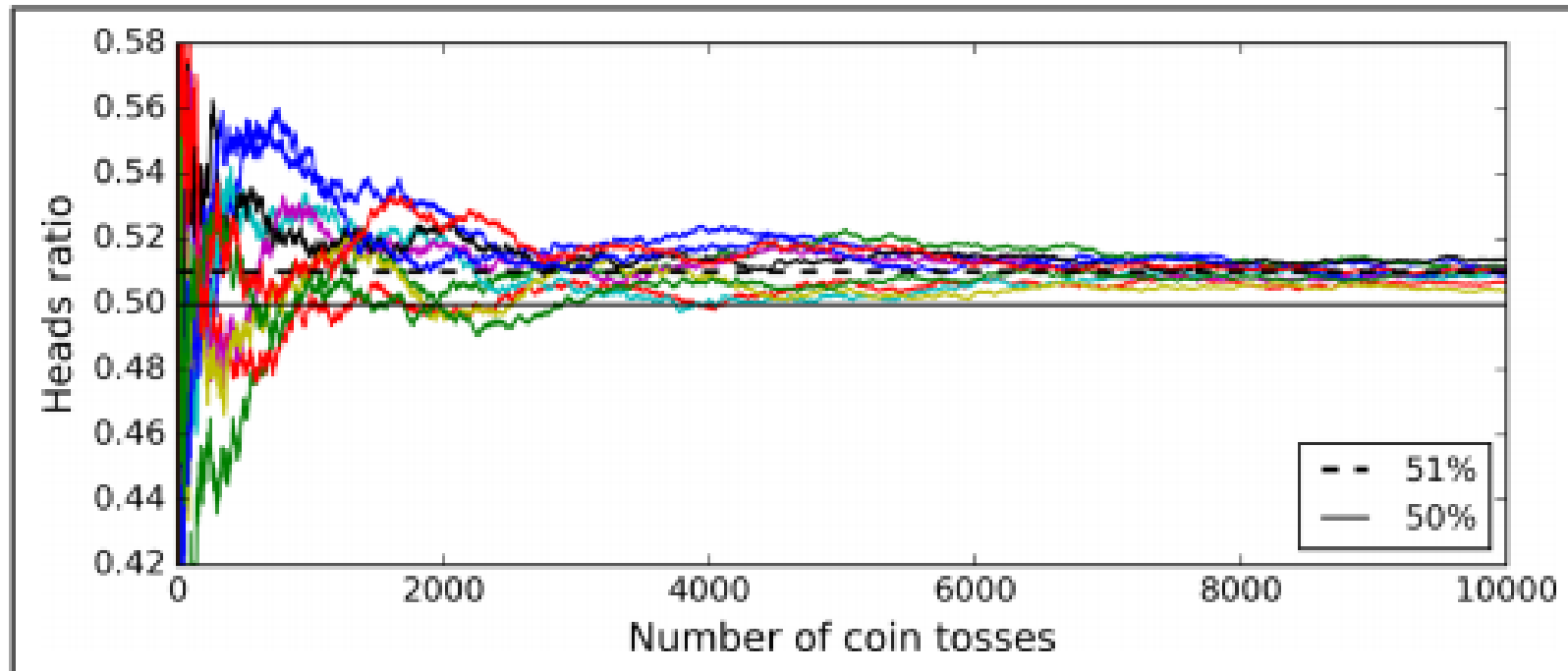


MODELOS DE ENSAMBLE

¿Cómo verificar que una moneda no esté trucada?

→ Lanzar la moneda n veces

→ A mayor número de lanzamientos, mejor la estimación



Géron, 2017



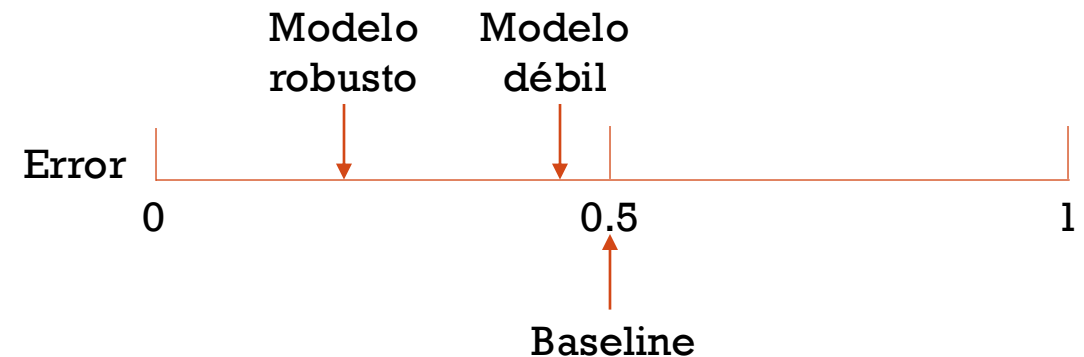
MODELOS DE ENSAMBLE

- Diversificación:
 - Cada modelo tiene un riesgo de mala predicción
 - ¿Por qué limitar la decisión a un solo modelo si podemos utilizar varios?
 - Combinar un grupo de clasificadores/regresores
 - Decisión basada en la agregación de varios modelos (max/promedio)
- Reducir la varianza global al **agregar** un conjunto de modelos supervisados, y obtener una mayor capacidad de generalización
- Se aumenta la calidad de la predicción, pero se pierde en interpretabilidad

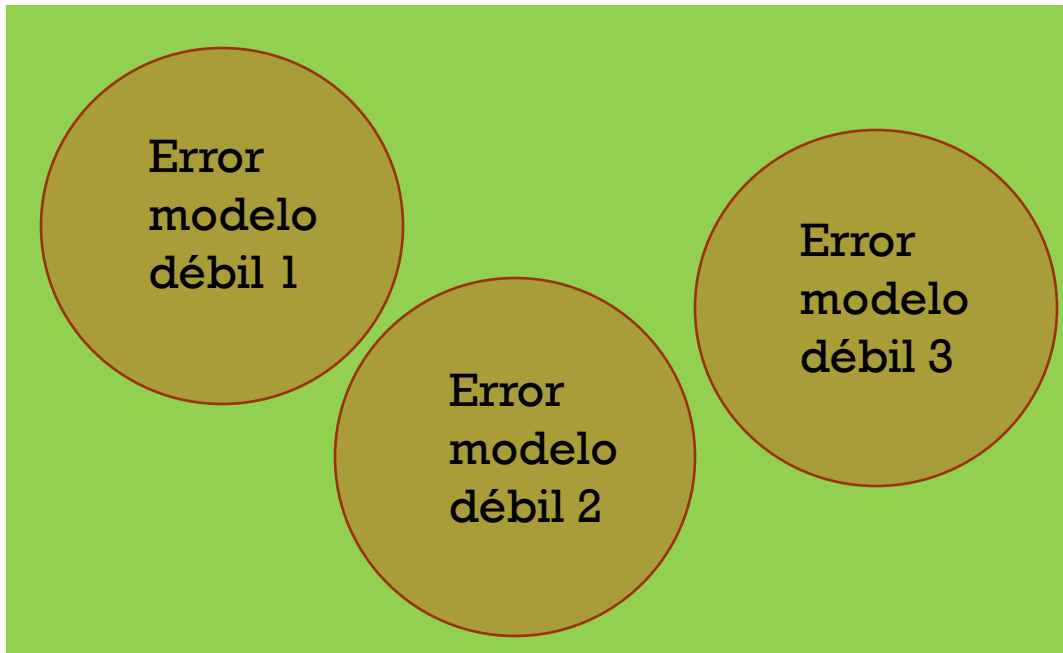


MODELOS DE ENSAMBLE

- Modelos débiles vs. robustos:
 - Los modelos robustos tenderán a estar de acuerdo en la mayoría de los casos
 - A menudo los datos permiten construir modelos que son apenas mejores al baseline



BAGGING (BOOTSTRAP AGGREGATING)



- ¿Cómo es nuestra predicción si ponemos a votar a 3 modelos?

→ Entre más de-correlacionados estén los modelos individuales, mejor va a ser el meta-modelo agregado



BAGGING

- Bagging = Bootstrap aggregating
- Bootstrap: técnica de muestreo
 - Consideración de varios conjuntos de entrenamiento/test utilizando **muestreo con reemplazo**
 - Por lo general los muestreos tienen el mismo tamaño del conjunto original
 - Reduce la varianza

Original Dataset

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

Bootstrap 1

x_8	x_6	x_2	x_9	x_5	x_8	x_1	x_4	x_8	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Bootstrap 2

x_{10}	x_1	x_3	x_5	x_1	x_7	x_4	x_2	x_1	x_8
----------	-------	-------	-------	-------	-------	-------	-------	-------	-------

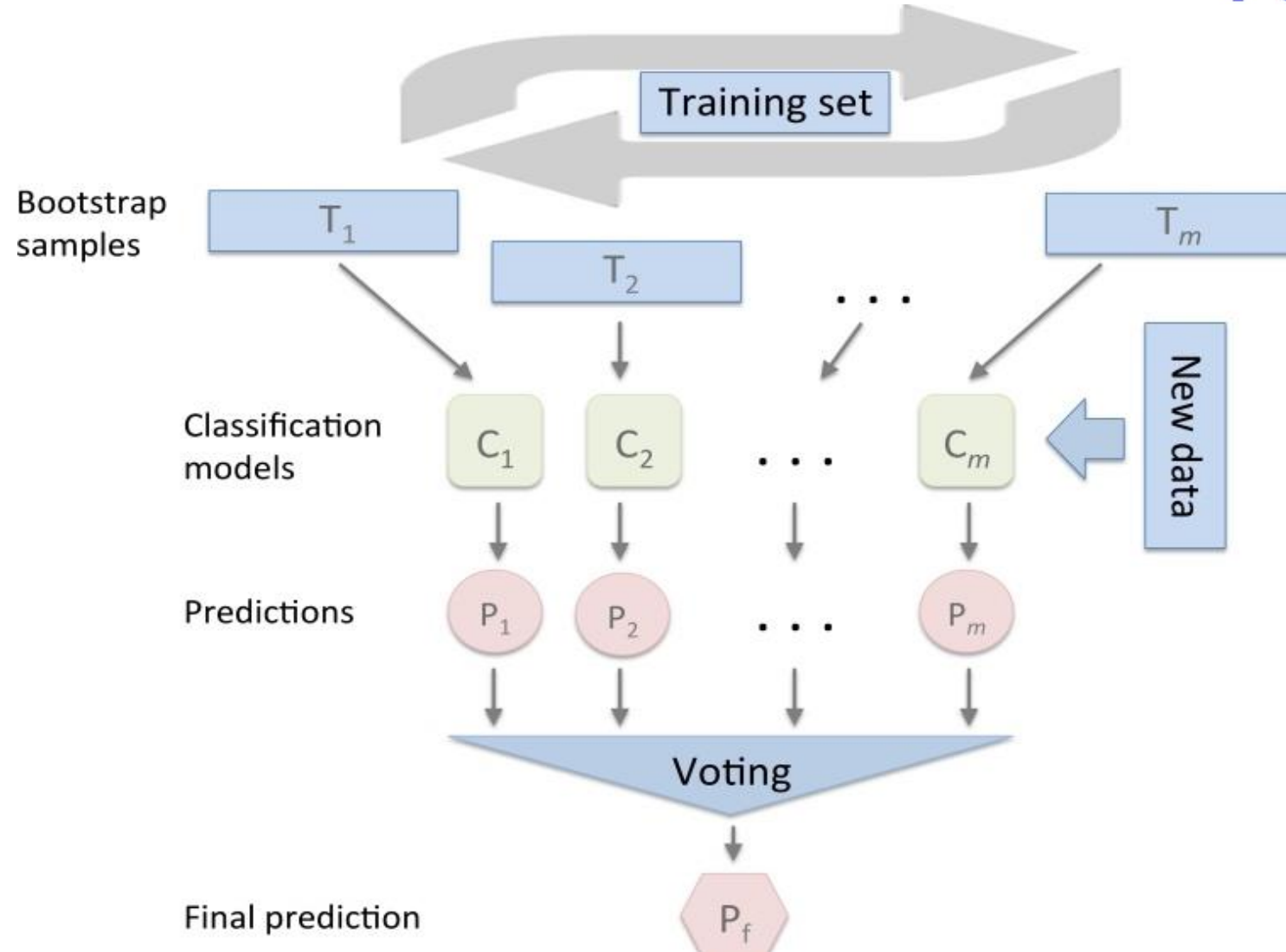
Bootstrap 3

x_6	x_5	x_4	x_1	x_2	x_4	x_2	x_6	x_9	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Sebastian Raschka, 2015



BAGGING



Raschka, 2017



BAGGING

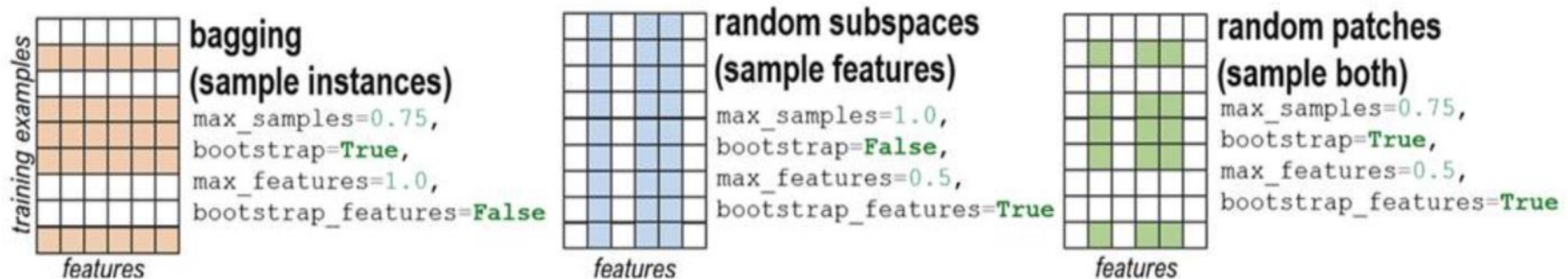
- Algoritmo (Breiman, 1994-1996):
 1. Escoger múltiples subconjuntos de instancias con repetición
 2. Entrenar varias instancias de un mismo tipo de modelo “**robusto**” de aprendizaje (bajo sesgo – alta varianza) basado en cada subconjunto
 3. Agregar las decisiones de cada modelo en una sola decisión global (votación/promedio)
- Muy útil cuando se trata de modelos no lineales
- Resultado: sesgo similar, reducción de varianza (estabilización de las predicciones) → mejora la exactitud
- Se disminuye la interpretabilidad



BAGGING

- Se aplica tanto a clasificadores como a regresores

Figure 2.9 Bagging compared to random subspaces and random patches.



- Random patches:** subconjuntos tanto de los registros como de las variables independientes

Random patching



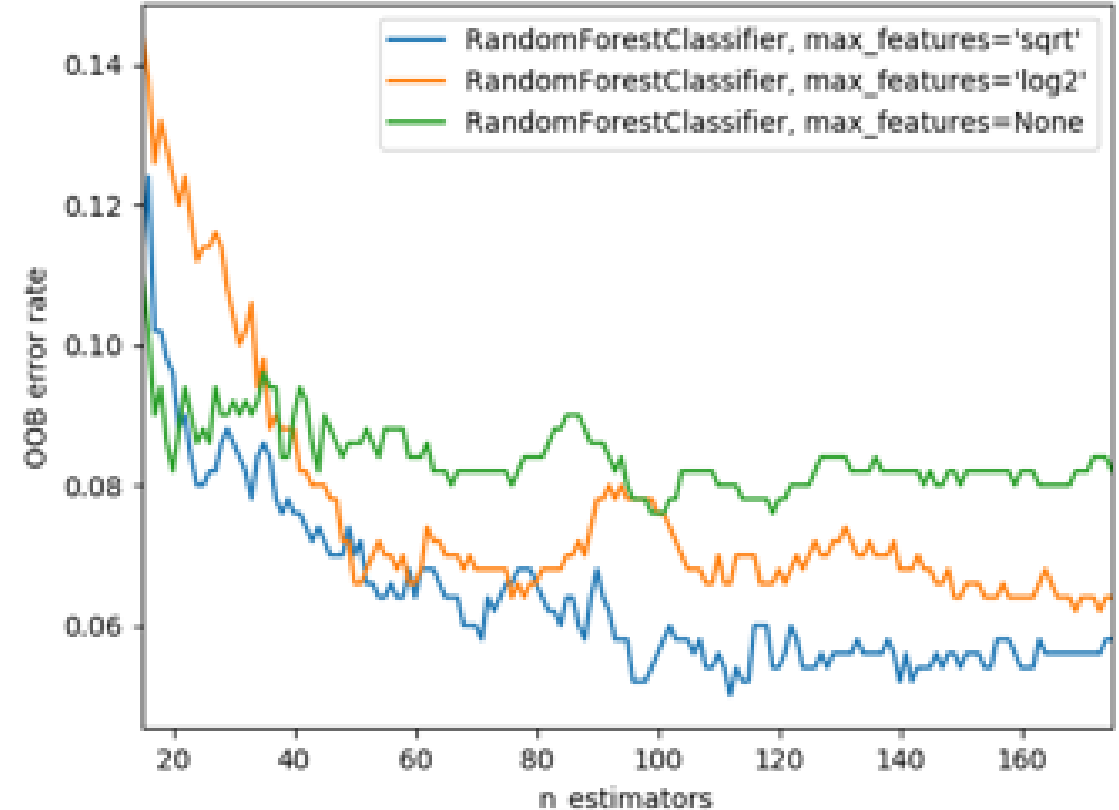
BAGGING

OOB (Out of bag error estimation):

- Un subconjunto bootstrap contiene, en promedio, aproximadamente $2/3$ de los datos del conjunto original
- OOB usa el $1/3$ restante de datos para calcular métricas de evaluación de los modelos individuales, que luego deben agregarse
- Con una cantidad elevada de árboles, OOB tiende hacia el error LOOCV :
 - Evita requerir de un test set
 - Evita el costoso proceso de K-Fold CV



- Junto a boosting, tiene un mejor rendimiento en caso de usar datos **estructurados**
- Extensión de bagging basado en árboles CART
- Cada árbol es **independiente** y tiene la **misma importancia** en la decisión final
- La idea es minimizar la **correlación** entre los modelos, forzando la diferencia más allá del bootstrap de los datos de entrenamiento
- Limita el **número m de atributos** a considerar en cada nodo de particionamiento.
- Valores por defecto:
 - En caso de regresión: $m/3$
 - En caso de clasificación: $\sqrt{m}$



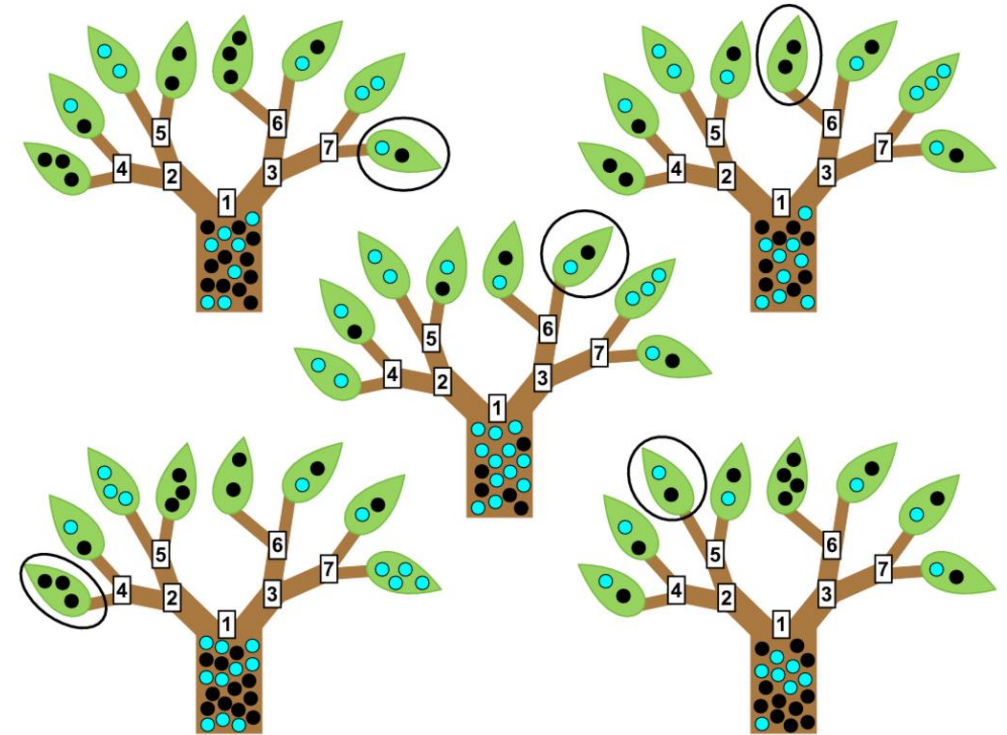
Scikit-learn.org



RANDOM FOREST

■ Algoritmo

1. Crear múltiples árboles de decisión, contruidos sobre muestras con reemplazo (bootstrapping) del dataset
2. En cada punto de partición, considerar solo una muestra aleatoria de todas las variables disponibles
3. Crecer el árbol
4. Agregar los resultados de todos los árboles creados



<http://inspirehep.net/record/1335130/plots>

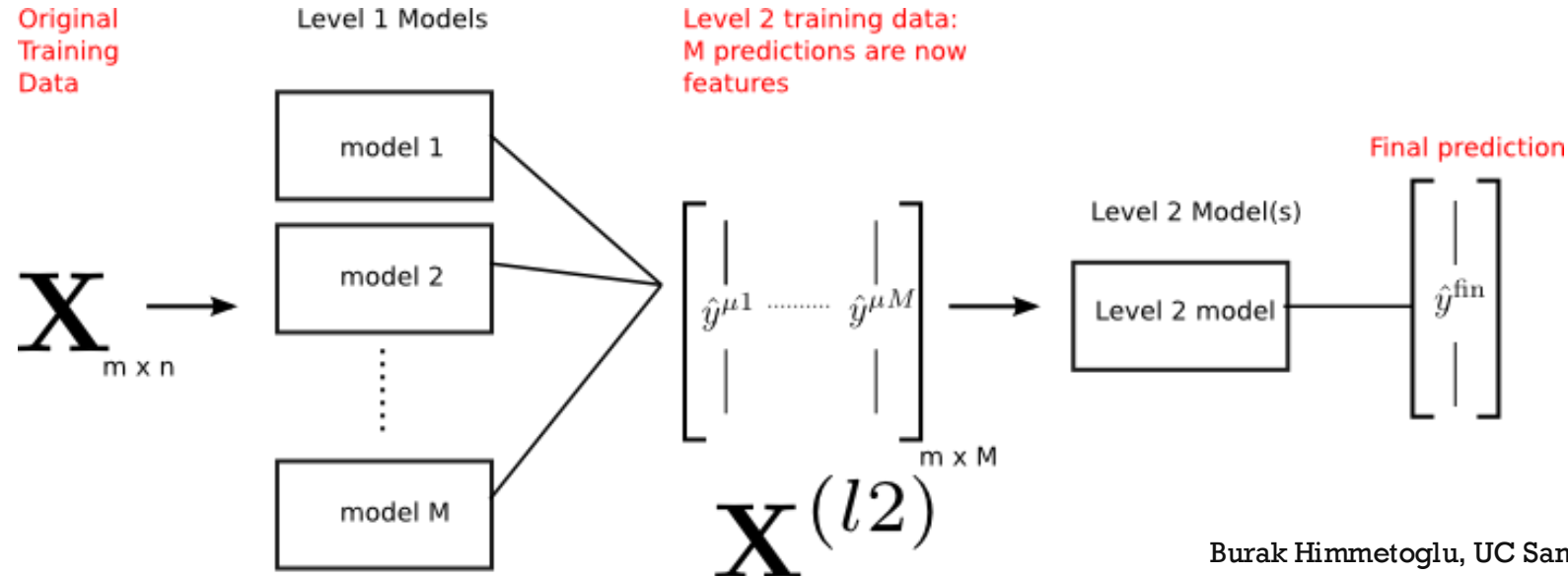


RANDOM FOREST

- Consideraciones
 - Al igual que el bagging:
 - **Heredan** cosas buenas de los árboles de decisión (importancia de las variables), mejoran las malas (varianza)
 - **Lento** (gran número de árboles a entrenar) y toman mucho espacio en memoria, pero es **escalable** (independencia)
 - No genera **overfitting** con más árboles, pero los árboles individuales pueden tenerlo (limitar su profundidad)
 - Muy difíciles de interpretar
 - Requieren de un buen afinamiento de parámetros (bagging es más simple)



- 2 capas de modelos:
 - Una primera capa incluye modelos diferentes, no necesariamente débiles, que se entrenan con el dataset original
 - Una segunda capa con un solo modelo, típicamente un modelo de regresión logística (clasificación) o un modelo lineal (regresión), que se entrena con las salidas de los modelos de la primera capa
 - Los modelos de las 2 capas comparten la misma variable objetivo

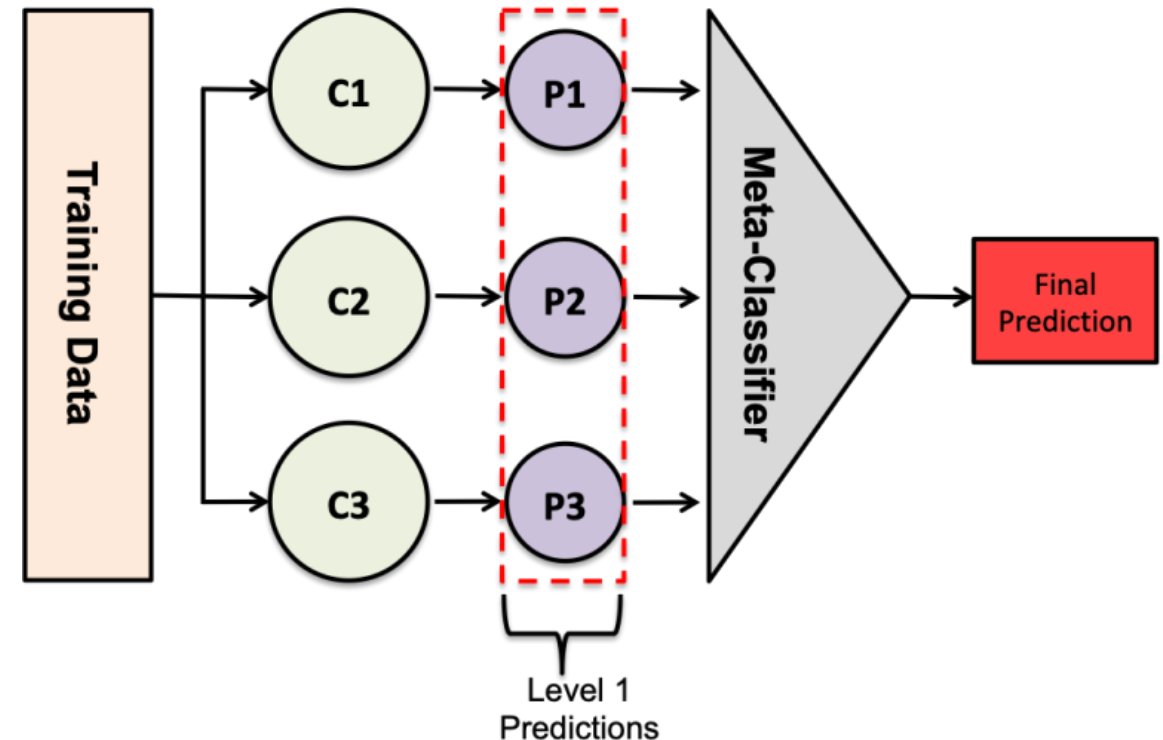


Burak Himmetoglu, UC Santa Barbara.



STACKING

- Combina varios estimadores para reducir sus sesgos.
- Las predicciones de cada estimador se apilan y se usan como entrada para un estimador final
- El entrenamiento del estimador final se realiza mediante validación cruzada.
- Se usa en problemas de regresión o de clasificación.



* C1, C2, and C3 are considered level 1 classifiers.

Fuente: neptune.ai



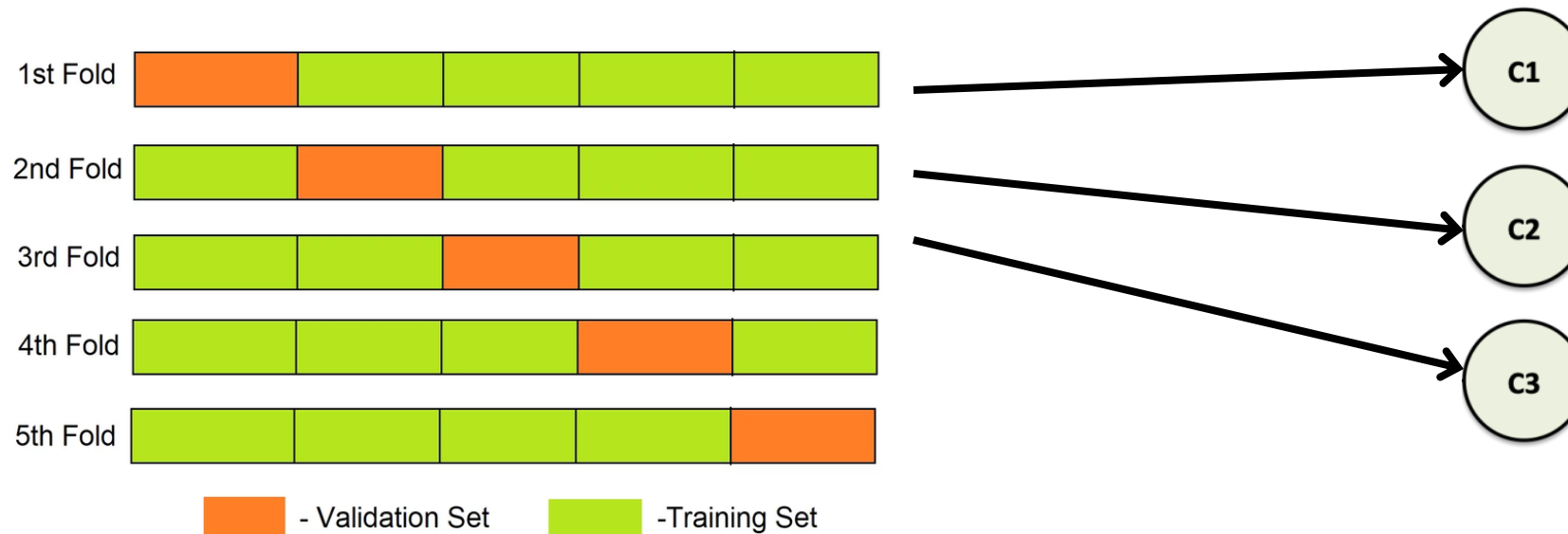
STACKING

1

El dataset original se divide en un k-fold CV

2

Cada k-fold es asignado a un clasificador de nivel 1 para sus respectivas predicciones.



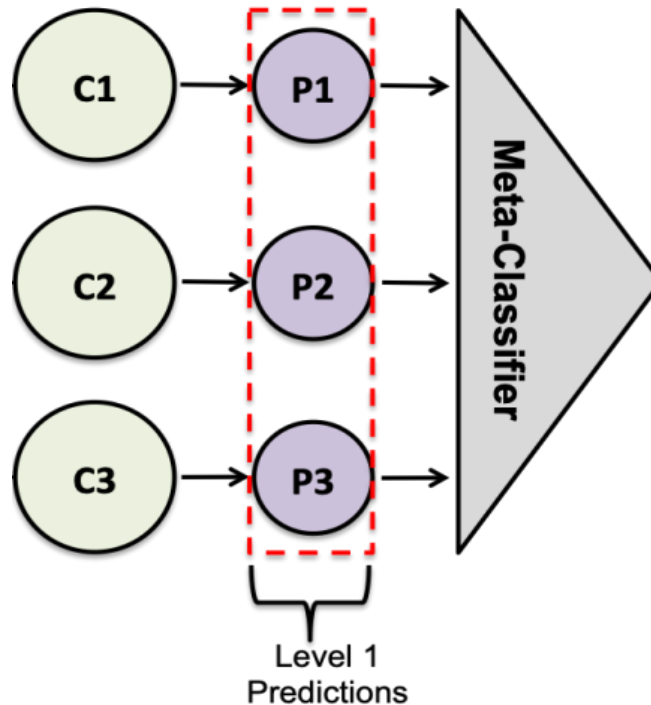
Fuente: www.askpython.com

* C1, C2, and C3 are considered level 1 classifiers.



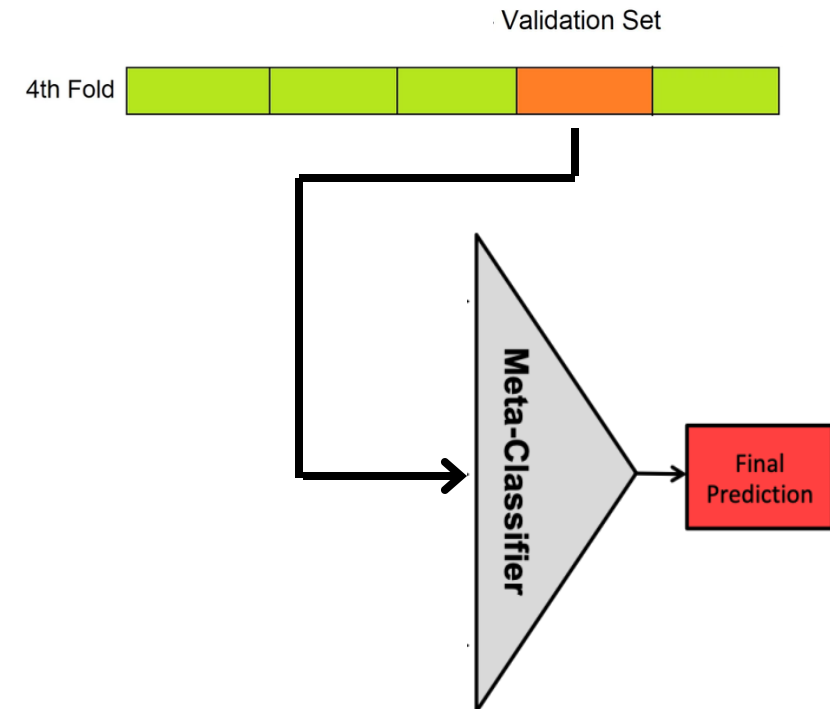
3

Las predicciones de nivel 1 son utilizadas para el entrenamiento del meta-clasificador.



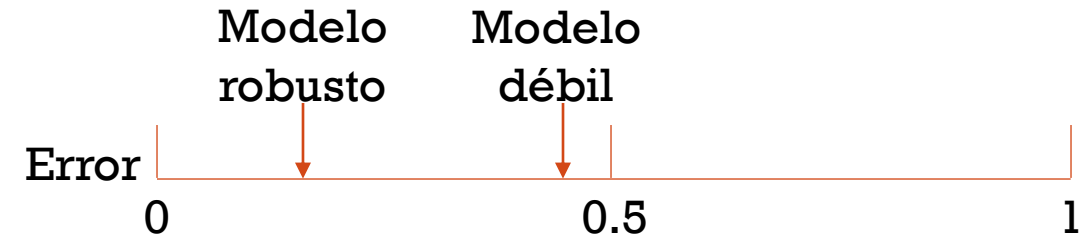
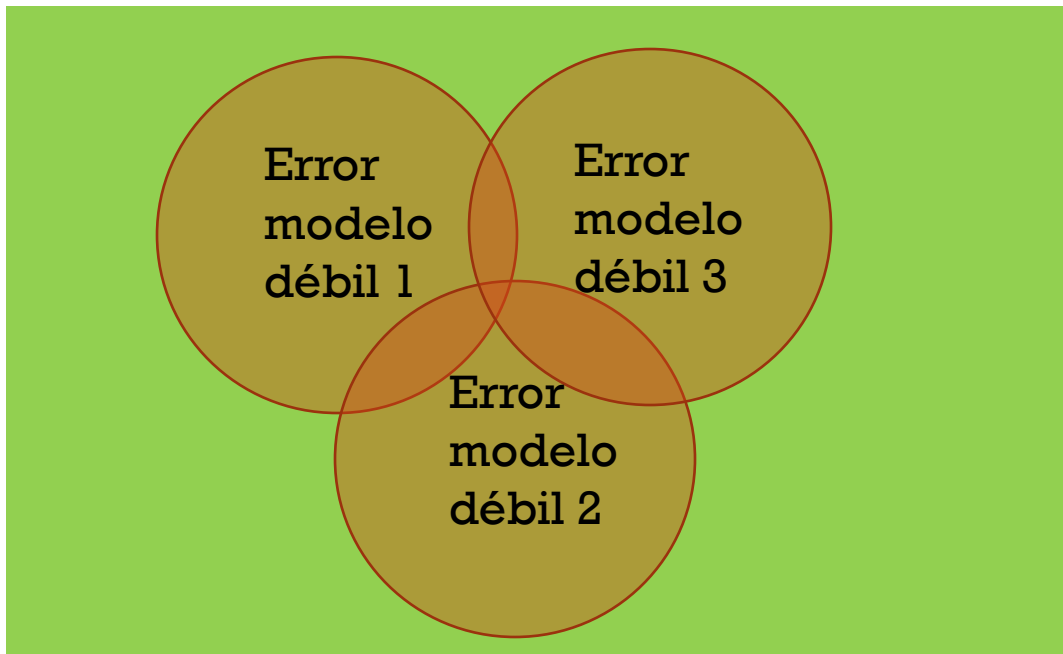
4

El meta-clasificador es probado con datos del conjunto de pruebas



BOOSTING

- Modelos débiles vs. robustos



- ¿Hay ciertas regiones más importantes en cuanto al error que otras?
- ¿Cómo mejoro en las regiones más susceptibles al error donde ningún modelo logra buenos resultados?



BOOSTING

- Agregación de predictores **débiles**, aprendidos **secuencialmente**
- Aprendizaje lento **progresivo**: reduce el error de sesgo al considerar los errores previos en los modelos subsiguientes.
- La **importancia de cada registro** se modifica según el error del último modelo entrenado → Cada modelo **depende** de los modelos anteriores
- Cada modelo tiene **diferente influencia** en la decisión final
- La idea es reducir el error de sesgo al considerar los errores previos en los modelos subsiguientes
- Puede caer en **overfitting** si el número de árboles (modelos) es grande



■ Algoritmo

1. Crear un modelo predictor **débil** inicial sobre el dataset
2. Asignar un **peso** a la decisión del modelo basado en su **error** de entrenamiento
3. Actualización de los pesos de cada instancia del dataset: se aumenta en las instancias que fueron mal predichas, se reduce en las instancias correctas
4. Crear un nuevo predictor, influenciado por los errores cometidos por los modelos anteriores
5. Agregar una decisión ponderada de todos los predictores creados

■ Tres parámetros importantes:

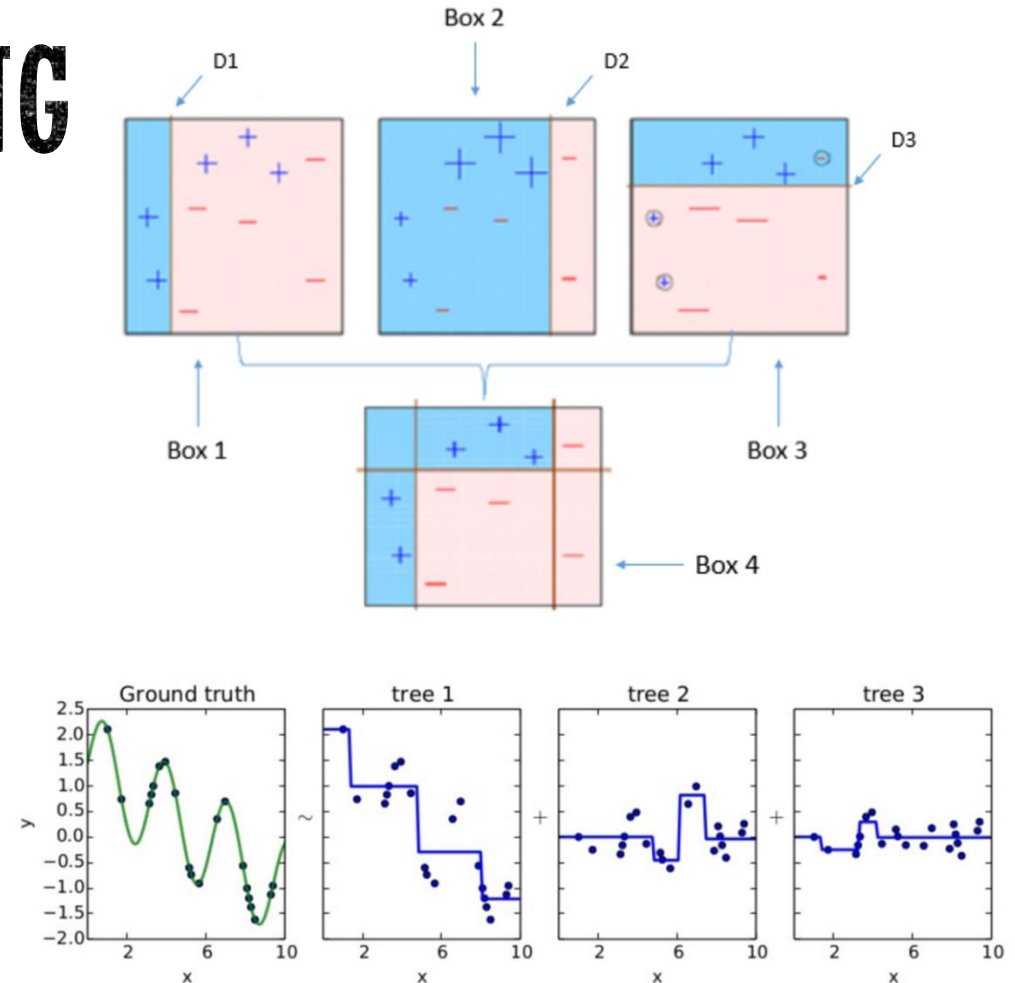
- **Número de árboles** a desarrollar
- **Learning rate o shrinking parameter**: controla la velocidad del aprendizaje para limitar el overfitting (valores típicos 0.01 - 0.001)
- **Número de particiones**: Controla la complejidad de los árboles. Se conoce como interacción de profundidad máxima (1 funciona bien)



VARIANTES DE BOOSTING

- ADA BOOST
- GRADIENT BOOSTING
- EXTREME GRADIENT BOOSTING (XGBOOST)

■ Hands-On Machine Learning with Scikit-Learn and TensorFlow. Aurélien Géron. 2017. O'Reilly Medi

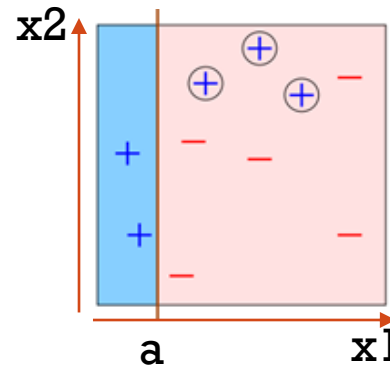
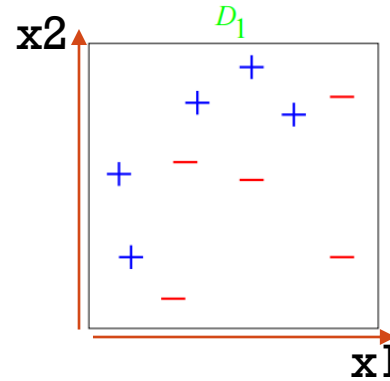


Source: [Quora](#)

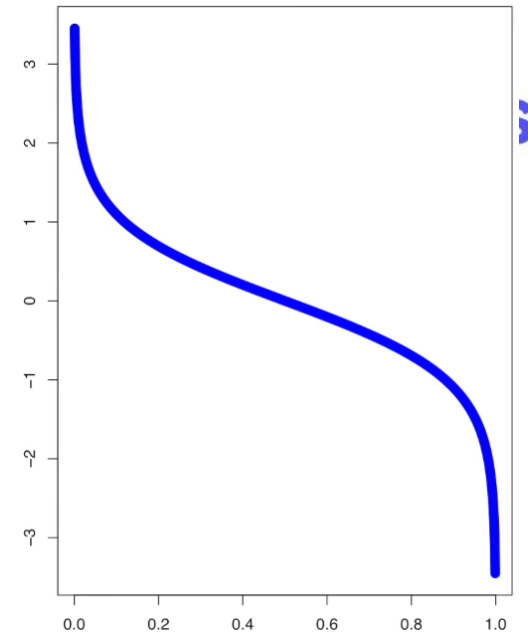
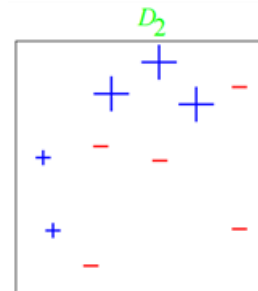


ADA BOOST

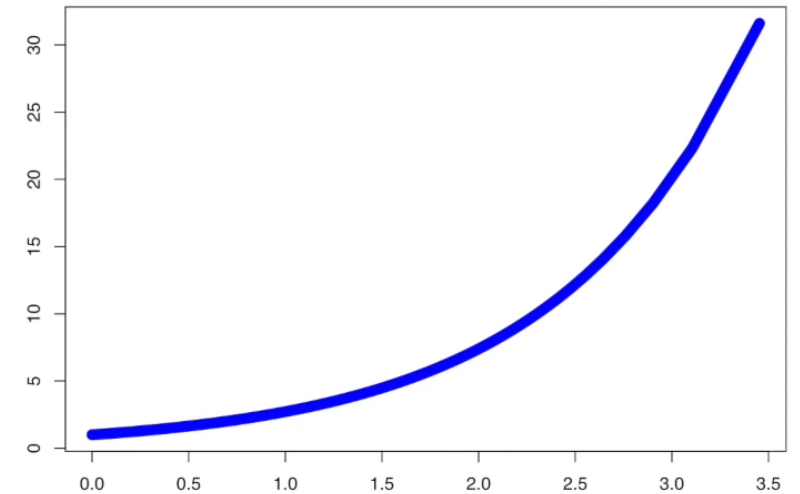
- Combinación de varios **decision stumps**, con **importancias diferentes** en la agregación final
- Inicialmente todas las instancias tienen el mismo peso: $1/n$
- Se encuentra el mejor decisión stump,
- Se evalúan los errores, se calcula la importancia del stump, y se define el **learning rate**: $\alpha = \frac{1}{2} \ln \frac{1-\text{error}}{\text{error}}$
- Actualizar los pesos de las instancias
 - Incorrectas: $\text{peso}_t = \text{peso}_{t-1} * e^{\alpha}$
 - Correctas: $\text{peso}_t = \text{peso}_{t-1} * e^{-\alpha}$



Árbol	Error	α
$x_1 < a$	3/10	0.424



α según el error



Nuevo peso dado α



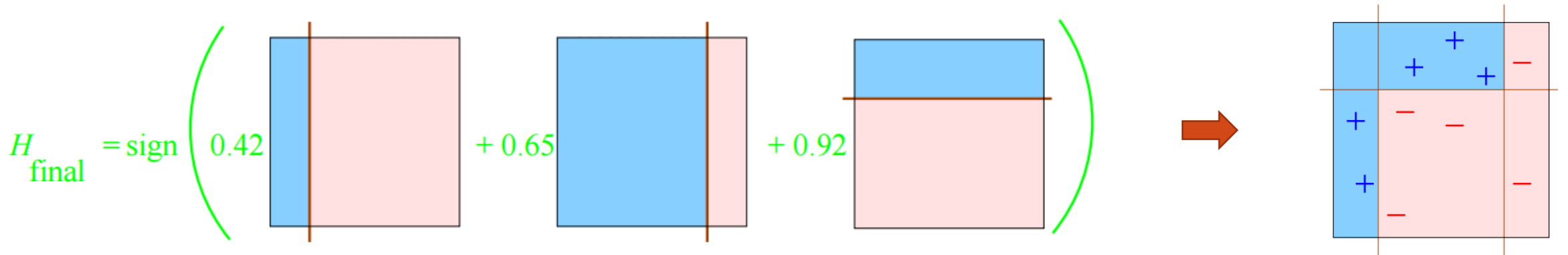
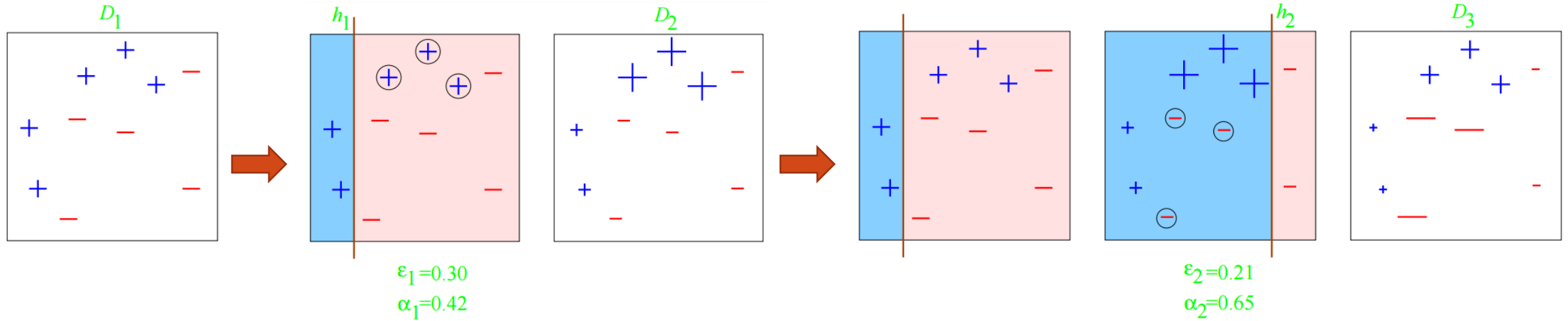


Universidad

ICESI

ADA BOOST

Posgrados



www.cs.princeton.edu/~schapire



TALLER: BAGGING, RANDOM FOREST, BOOSTING

- Vamos a analizar diferentes configuraciones de bagging y random forest para crear un clasificador del conjunto de datos churn.csv, considerando diferentes valores de los hiper-parámetros #árboles y max_depth
- Continuar con el notebook S3_Arboles_Churn desde la sección de ensambles
- Revisar los cuadernos S3_DecisionStump_Adaboost y S3_Práctica_Stacking



GRADIENT BOOSTING (REGRESIÓN)

Input: Data $\{(x_i, y_i)\}_{i=1}^n$, función de pérdida diferenciable $L(y_i, F(x_i))$.

Para regresión: $L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2$

1. Inicializar el modelo con un valor constante $F_0(x_i) = \operatorname{argmin}_{\theta} \sum_{i=1}^n L(y_i, \theta)$

$$\sum_{i=1}^n -(y_i - \theta) = 0 \rightarrow \theta * n = \sum_{i=1}^n y_i \rightarrow \theta = \bar{y} \text{ (promedio)}$$

2. Para $m=1$ a M :

- a) Calcular el **gradiente** (los pseudo residuos de cada instancia respecto a la predicción anterior)

$$r_{im} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} = -(-(y_i - F_{m-1}(x_i))) = y_i - F_{m-1}(x_i)$$

- b) Crear un árbol para predecir los r_{im} a partir de las variables independientes y crear sus J_m hojas R_{jm} , con $j = 1 \dots J_m$

- c) Estimar el valor de salida (**out**) hoja R_{jm} , con $j = 1 \dots J_m$ (como en el punto 1)

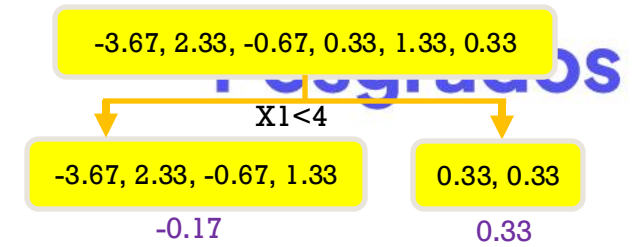
$$\theta_{jm} = \operatorname{argmin}_{\theta} \sum_{x_i \in R_{jm}} L(\theta, F_{m-1}(x_i) + \theta)$$

$$\sum_{x_i \in R_{jm}} -(y_i - F_{m-1}(x_i) - \theta) = 0 \rightarrow \theta * n = \sum_{x_i \in R_{jm}} (y_i - F_{m-1}(x_i)) \rightarrow \theta = \overline{r_{im}}$$

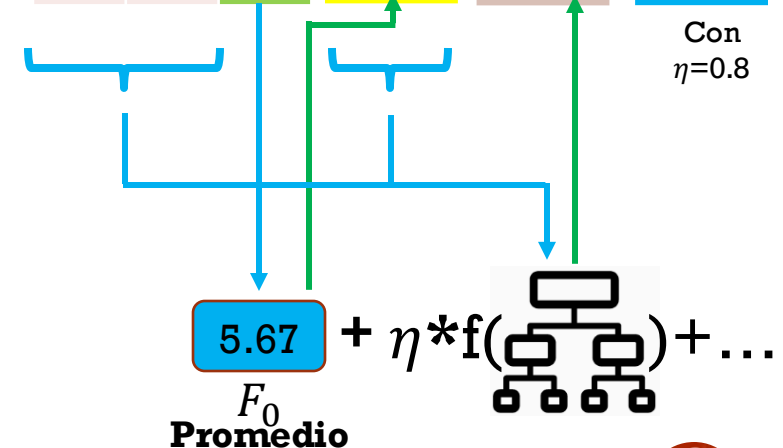
- d) Se actualiza la predicción con incorporando el nuevo modelo con un learning rate η :

$$F_m = F_{m-1}(x_i) + \eta * r_{im}$$

Se itera



X1	X2	Y	res	out	pred
1	..	2	-3.67	-0.17	5.53
2	..	8	2.33	-0.17	5.53
3	..	5	-0.67	-0.17	5.53
4	..	6	0.33	0.33	5.93
0	..	7	1.33	-0.17	5.53
5	..	6	0.33	0.33	5.93



GRADIENT BOOSTING (CLASIFICACIÓN)

- Queremos minimizar la función de pérdida (igual a maximizar el log likelihood):

$$\begin{aligned} L(y_i, F(x_i)) &= -(y_i * \log(F(x_i)) + (1 - y_i) * \log(1 - F(x_i))) \\ &= -(y_i * \log(p) + (1 - y_i) * \log(1 - p)) \\ &= -y_i * \log(p) - \log(1 - p) + y_i * \log(1 - p) \\ &= -y_i * (\log(p) - \log(1 - p)) - \log(1 - p) \\ &= -y_i * \log\left(\frac{p}{1-p}\right) - \log\left(1 - \frac{e^{\log\left(\frac{p}{1-p}\right)}}{1+e^{\log\left(\frac{p}{1-p}\right)}}\right) \\ &= -y_i * \log(\text{odds}) - \log\left(1 - \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}}\right) \\ &= -y_i * \log(\text{odds}) - \log\left(\frac{1+e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} - \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}}\right) \\ &= -y_i * \log(\text{odds}) - \log\left(\frac{1}{1+e^{\log(\text{odds})}}\right) \\ &= -y_i * \log(\text{odds}) + \log(1 + e^{\log(\text{odds})}) \end{aligned}$$

$$\begin{aligned} p &= \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} \\ 1 - p &= 1 - \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} \\ &= \frac{1+e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} - \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} \\ &= \frac{1}{1+e^{\log(\text{odds})}} \end{aligned}$$



GRADIENT BOOSTING (CLASIFICACIÓN)

- Buscamos el gradiente de la función de pérdida

$$L(y_i, \log(odds)) = -y_i * \log(odds) + \log(1 + e^{\log(odds)})$$

$$\frac{\partial(-y_i * (\log(odds)) + \log(1 + e^{\log(odds)}))}{\partial(\log(odds))} = -y_i + \frac{1}{1 + e^{\log(odds)}} * e^{\log(odds)}$$

$$= -y_i + \frac{e^{\log(odds)}}{1 + e^{\log(odds)}}$$

$$= -y_i + p$$

$$\begin{aligned} p &= \frac{e^{\log(odds)}}{1 + e^{\log(odds)}} \\ 1 - p &= 1 - \frac{e^{\log(odds)}}{1 + e^{\log(odds)}} \\ &= \frac{1 + e^{\log(odds)}}{1 + e^{\log(odds)}} - \frac{e^{\log(odds)}}{1 + e^{\log(odds)}} \\ &= \frac{1}{1 + e^{\log(odds)}} \end{aligned}$$



GRADIENT BOOSTING (CLASIE)

Input: Data $\{(x_i, y_i)\}_{i=1}^n$, función de pérdida diferenciable $L(y_i, F(x_i))$.

$$L(y_i, \log(odds)) = -y_i * \log(odds) + \log(1 + e^{\log(odds)})$$

1. Inicializar el modelo con un valor constante

$F_0(x_i) = \operatorname{argmin}_{\theta} \sum_{i=1}^n L(y_i, \theta)$, se usa el gradiente

$$\sum_{i=1}^n -y_i + \frac{1}{1+e^{\theta}} * e^{\theta} = 0 \rightarrow \frac{1}{1+e^{\theta}} * e^{\theta} * n = \sum_{i=1}^n y_i$$

$$\rightarrow \frac{e^{\theta}}{1+e^{\theta}} = p \rightarrow \theta = \log(odds) \quad (p \equiv \text{probability score})$$

$$F_0(x_i) = \log(odds)$$

2. Para $m=1$ a M :

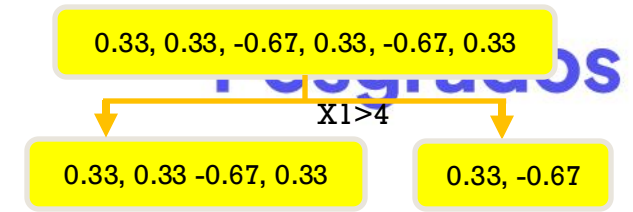
- a) Calcular los pseudo residuos de cada instancia con respecto a la predicción anterior (el **gradiente**)

$$r_{im} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} = -(-y_i + p) = y_i - p$$

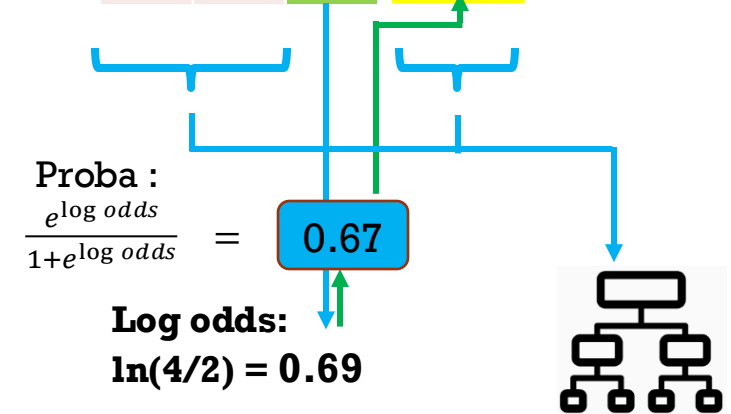
- b) Crear un árbol para predecir los r_{im} a partir de las variables independientes y crear sus J_m hojas R_{jm} , con $j = 1 \dots J_m$

- c) Estimar el valor de salida hoja R_{jm} , con $j = 1 \dots J_m$

$$\theta_{jm} = \operatorname{argmin}_{\theta} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \theta)$$



X1	X2	Y	res
1	..	1	0.33
2	..	1	0.33
3	..	0	-0.67
4	..	1	0.33
5	..	0	-0.67
6	..	1	0.33



GRADIENT BOOSTING (CLASIFICACIÓN)

- Para obtener el valor de salida de cada hoja necesitamos derivar $L(y_i, F_{m-1}(x_i) + \theta)$

$$L(y_i, F_{m-1}(x_i) + \theta) = -y_i * \log(F_{m-1}(x_i) + \theta) + \log(1 + e^{F_{m-1}(x_i) + \theta})$$

Se aproxima la función de pérdida con un polinomio de Taylor de orden 2:

$$L(y_i, F_{m-1}(x_i) + \theta) \approx L(y_i, F_{m-1}(x_i)) + \frac{\partial}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i)) * \theta + \frac{1}{2} \frac{\partial^2}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i)) * \theta^2$$

Y ahora si se deriva con respecto a θ , para optimizarlo:

$$\frac{\partial}{\partial \theta} L(y_i, F_{m-1}(x_i) + \theta) \approx \frac{\partial}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i)) + \frac{\partial^2}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i)) * \theta = 0$$

$$\theta = \frac{-\frac{\partial}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i))}{\frac{\partial^2}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i))} = \frac{y_i - p}{\frac{\partial^2}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i))}$$



GRADIENT BOOSTING (CLASIFICACIÓN)

- Necesitamos obtener la derivada segunda de la función de pérdida

$$\begin{aligned}
 \frac{\partial^2}{\partial \log(odds)^2} L(y_i, \log(odds)) &= \frac{\partial}{\partial \log(odds)} \left(-y_i + \frac{1}{1+e^{\log(odds)}} * e^{\log(odds)} \right) \\
 &= \frac{-e^{\log(odds)}}{(1+e^{\log(odds)})^2} * e^{\log(odds)} + \frac{1}{1+e^{\log(odds)}} * e^{\log(odds)} \\
 &= \frac{-e^{2*\log(odds)}}{(1+e^{\log(odds)})^2} + \frac{e^{\log(odds)}}{1+e^{\log(odds)}} \\
 &= \frac{-e^{2*\log(odds)}}{(1+e^{\log(odds)})^2} + \frac{e^{\log(odds)} + e^{2*\log(odds)}}{(1+e^{\log(odds)})^2} \\
 &= \frac{e^{\log(odds)}}{(1+e^{\log(odds)})^2} = \frac{e^{\log(odds)}}{1+e^{\log(odds)}} * \frac{1}{1+e^{\log(odds)}} = p * (1 - p)
 \end{aligned}$$

- Por lo tanto, considerando todas las instancias de un nodo tenemos $\theta_{jm} = \operatorname{argmin}_{\theta} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \theta)$,

que se logra cuando $\theta = \frac{y_i - p}{\frac{\partial^2}{\partial F_{m-1}(x_i)} L(y_i, F_{m-1}(x_i))} = \frac{y_i - p}{p * (1 - p)}$

$$\theta_{jm} = \frac{\sum_{i \in R_{jm}} (y_i - p)}{\sum_{i \in R_{jm}} p(1 - p)}$$



GRADIENT BOOSTING (CLASIE)

Input: Data $\{(x_i, y_i)\}_{i=1}^n$, función de pérdida diferenciable $L(y_i, F(x_i))$.
 $L(y_i, \log(odds)) = -y_i * \log(odds) + \log(1 + e^{\log(odds)})$

1. Inicializar el modelo con un valor constante

$F_0(x_i) = \operatorname{argmin}_{\theta} \sum_{i=1}^n L(y_i, \theta)$, se usa el gradiente

$$\sum_{i=1}^n -y_i + \frac{1}{1+e^{\theta}} * e^{\theta} = 0 \rightarrow \frac{1}{1+e^{\theta}} * e^{\theta} * n = \sum_{i=1}^n y_i$$

$$\rightarrow \frac{e^{\theta}}{1+e^{\theta}} = p \rightarrow \theta = \log(odds)$$

$$F_0(x_i) = \log(odds)$$

2. Para $m=1$ a M :

- a) Calcular los pseudo residuos de cada instancia con respecto a la predicción anterior (el **gradiente**)

$$r_{im} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} = -(-y_i + p) = y_i - p$$

- b) Crear un árbol para predecir los r_{im} a partir de las variables independientes y crear sus J_m hojas R_{jm} , con $j = 1 \dots J_m$

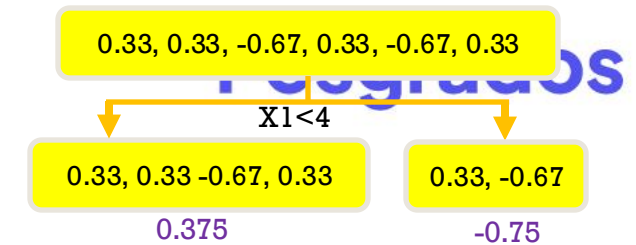
- c) Estimar el valor de salida hoja R_{jm} , con $j = 1 \dots J_m$

$$\theta_{jm} = \operatorname{argmin}_{\theta} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \theta) = \frac{\sum y_i - p}{\sum p * (1-p)}$$

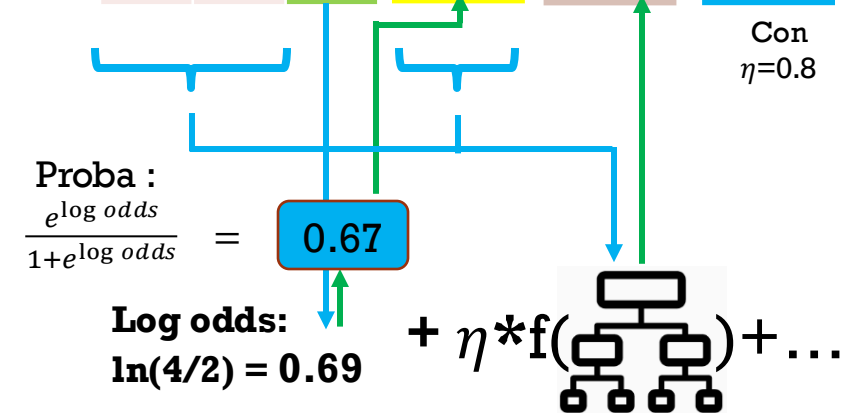
$$\frac{0.33 - 0.67}{0.67 * 0.33 + 0.67 * 0.33} = -0.75$$

- d) Se actualiza la predicción con incorporando el nuevo modelo con un learning rate η :

$$F_m = F_{m-1}(x_i) + \eta * r_{im}$$



X1	X2	Y	res	out	pred
1	..	1	0.33	0.375	0.967
2	..	1	0.33	0.375	0.967
3	..	0	-0.67	0.375	0.967
4	..	1	0.33	0.375	0.967
5	..	0	-0.67	-0.75	0.067
6	..	1	0.33	-0.75	0.067



XGBOOST (REGRESIÓN)

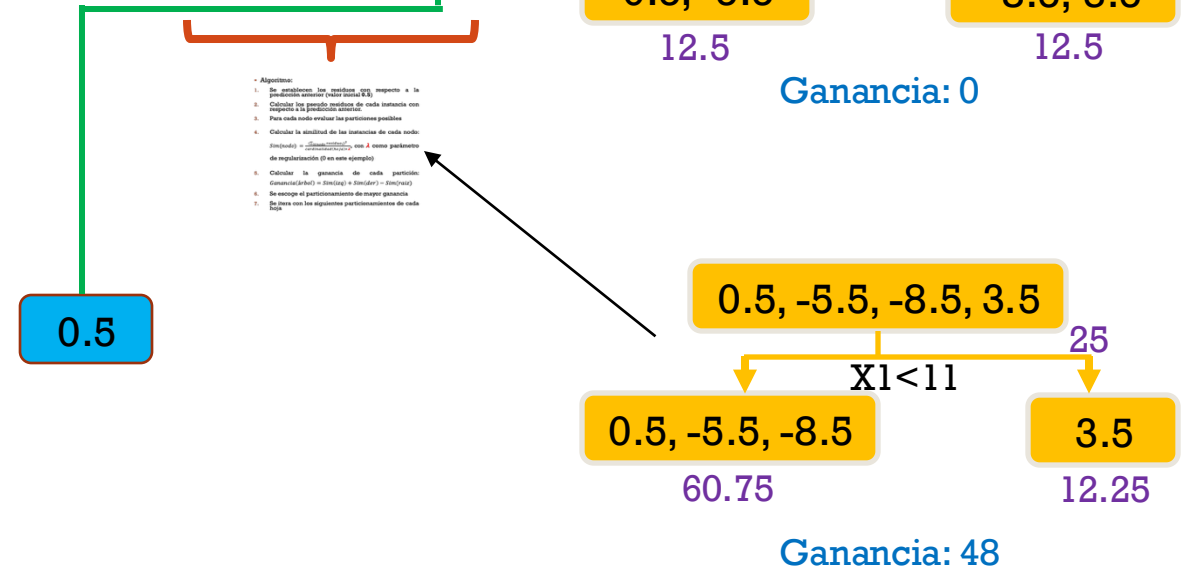
Algoritmo:

1. Se establecen los residuos con respecto a la predicción anterior (valor inicial 0.5)
2. Calcular los pseudo residuos de cada instancia con respecto a la predicción anterior.
3. Para cada nodo evaluar las particiones posibles
4. Calcular la similitud de las instancias de cada nodo:

$$Sim(nodo) = \frac{(\sum_{i \in nodo} residuo_i)^2}{cardinalidad(hoja) + \lambda}$$
 con λ como parámetro de regularización (0 en este ejemplo)
5. Calcular la ganancia de cada partición:

$$Ganancia(árbol) = Sim(izq) + Sim(der) - Sim(raiz)$$
6. Se escoge el particionamiento de mayor ganancia
7. Se itera con los siguientes particionamientos de cada hoja

X1	X2	Y	X1	X2	res
13	..	4	13	..	3.5
9	..	-8	9	..	-8.5
7	..	-5	7	..	-5.5
1	..	1	1	..	0.5



XGBOOST (REGRESIÓN)

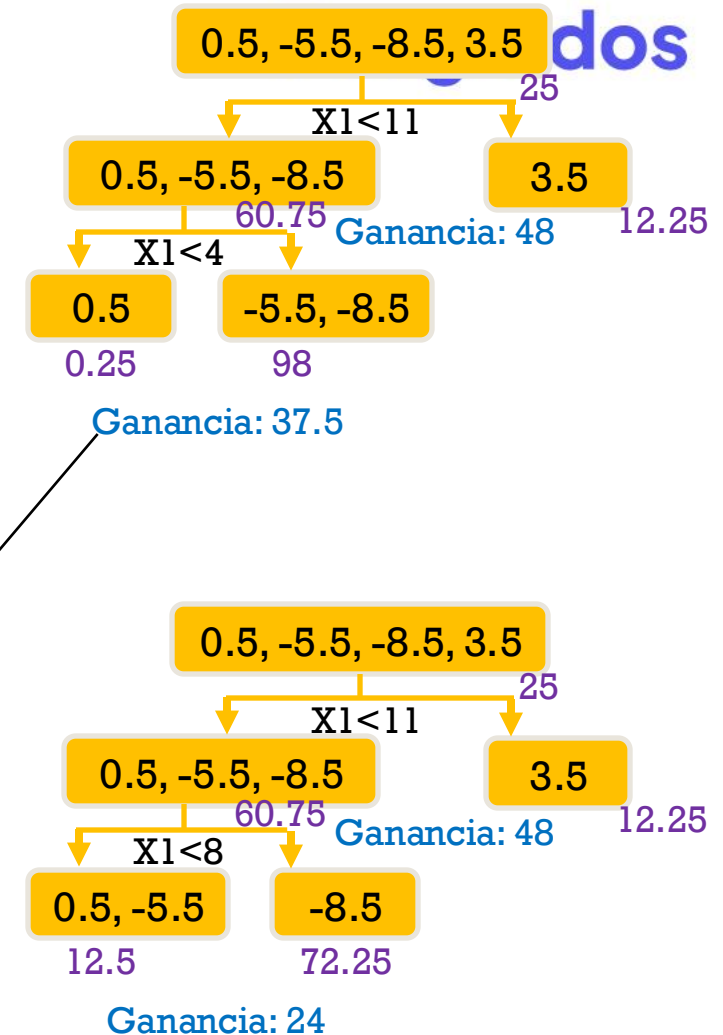
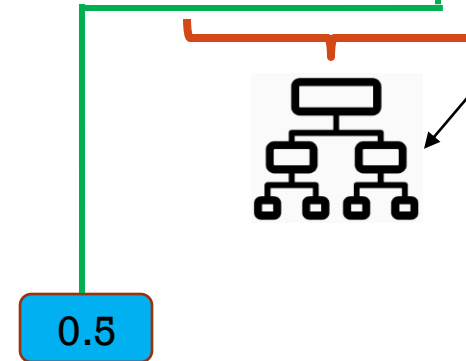
- Algoritmo (cont.)

- Se itera en los siguientes particionamientos por cada hoja hasta alcanzar un criterio de parada (e.g. **max depth**=2)
- Una vez desarrollado el árbol, se poda (post-pruning), teniendo en cuenta un umbral mínimo γ (**gamma**) de ganancia para particionar un nodo:

Ganancia $> \gamma$

$$Sim(nodo) = \frac{(\sum_{i \in nodo} residuo_i)^2}{cardinalidad(hoja) + \lambda}$$

X1	X2	Y	X1	X2	res
13	..	4	13	..	3.5
9	..	-8	9	..	-8.5
7	..	-5	7	..	-5.5
1	..	1	1	..	0.5



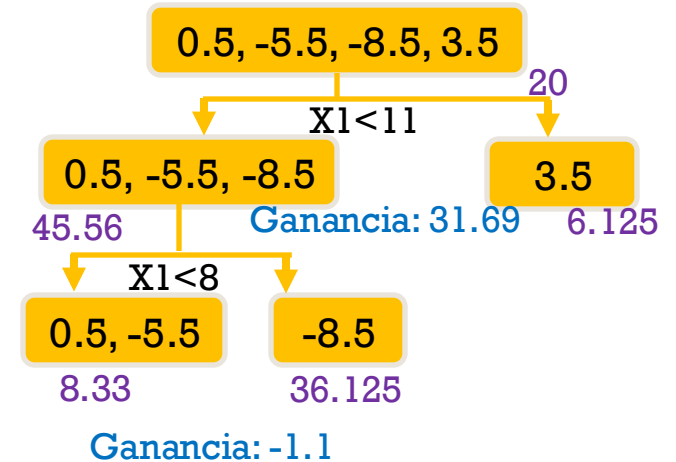
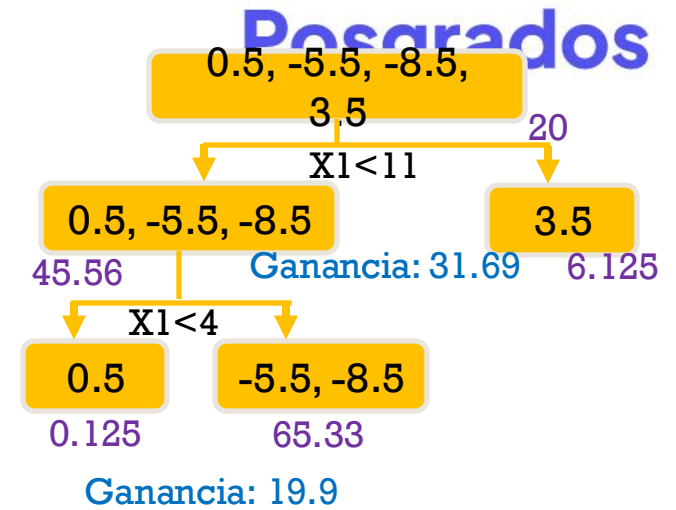
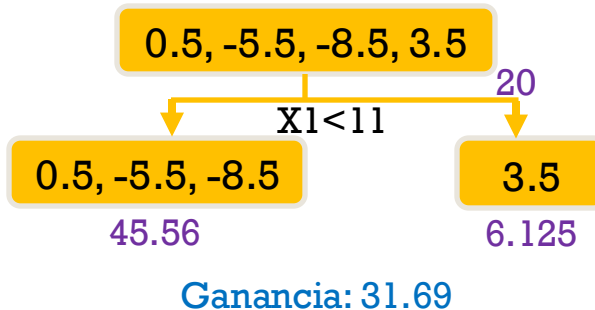
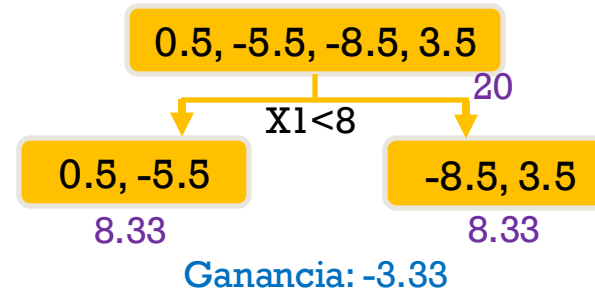
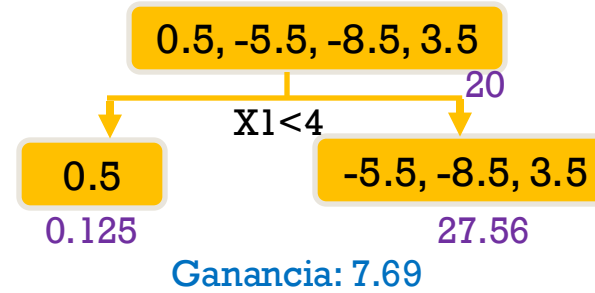
- Si $\gamma = 30$, ¿cómo sería el árbol?
- Si $\gamma = 40$, ¿cómo sería el árbol?
- Si $\gamma = 30$ y $\lambda = 1$, ¿cómo sería el árbol?



XGBOOST

$$Sim(nodo) = \frac{(\sum_{i \in nodo} residuo_i)^2}{cardinalidad(hoja) + \lambda}$$

- Si $\gamma = 30$ y $\lambda = 1$,
¿Cómo sería el árbol?



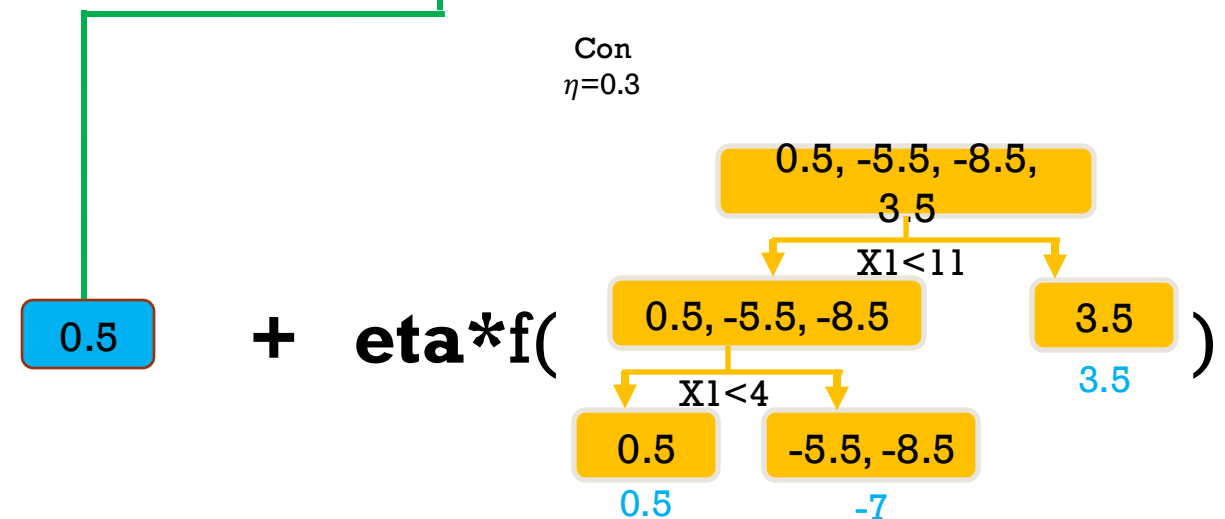
■ Algoritmo :

7. Se itera en los siguientes particionamientos por cada hoja hasta alcanzar un criterio de parada (e.g. **max depth**=2)
8. Una vez desarrollado el árbol, se poda (post-pruning), teniendo en cuenta un umbral mínimo γ (**gamma**) de ganancia para particionar un nodo.
9. Calculamos los valores predichos por el árbol nodo por nodo:

$$Pred(nodo) = \frac{\sum_{i \in nodo} residuo_i}{cardinalidad(hoja) + \lambda}$$
10. Combinamos el nuevo árbol usando un único **learning rate**, (llamado **eta**, por defecto 0.3)
11. Se crean nuevos árboles hasta alcanzar un criterio de parada (e.g. # árboles)

X1	X2	Y	X1	X2	res	PRED	res
13	..	4	13	..	3.5	1.55	2.45
9	..	-8	9	..	-8.5	-1.6	-6.4
7	..	-5	7	..	-5.5	-1.6	-3.4
1	..	1	1	..	0.5	0.65	0.35

...



- Para regresión se usa la función de pérdida $L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2$
 - Su gradiente **g** (primera derivada) es: $\mathbf{g} = \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = -(y_i - F(x_i))$, es decir el residuo negativo
 - Su hessiano **h** (segunda derivada) es: $\mathbf{h} = \frac{\partial^2 L(y_i, F(x_i))}{\partial F(x_i)^2} = \frac{\partial -(y_i - F(x_i))}{\partial F(x_i)} = 1$
- Para cada hoja buscamos la salida **O** que minimice $(\sum_{i=1}^n L(y_i, F_{m-1}(x_i) + O)) + \frac{1}{2}\lambda * O^2$, con **λ** para regularización
 - Usamos aproximación de Taylor de orden 2.

$$\sum_{i=1}^n L(y_i, F_{m-1}(x_i) + O) = \sum_{i=1}^n L(y_i, F(x_i)) + \sum_{i=1}^n \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} * O + \frac{1}{2} * \sum_{i=1}^n \frac{\partial^2 L(y_i, F(x_i))}{\partial F(x_i)^2} * O^2$$

- Vamos a minimizar con respecto a **O**

$$\begin{aligned} & \sum_{i=1}^n L(y_i, F(x_i)) + \sum_{i=1}^n \mathbf{g} * O + \frac{1}{2} * \sum_{i=1}^n \mathbf{h} * O^2 + \frac{1}{2} \lambda * O^2 \\ & \frac{\partial \sum_{i=1}^n L(y_i, F(x_i)) + \sum_{i=1}^n \mathbf{g} * O + \frac{1}{2} * \sum_{i=1}^n \mathbf{h} * O^2 + \frac{1}{2} \lambda * O^2}{\partial O} = 0 \\ & \sum_{i=1}^n \mathbf{g} + \sum_{i=1}^n \mathbf{h} * O + \lambda * O = 0 \\ & O = \frac{-\sum_{i=1}^n \mathbf{g}}{\sum_{i=1}^n \mathbf{h} + \lambda} = \frac{\text{suma de los residuos de la hoja}}{\text{cardinalidad de la hoja} + \lambda} \end{aligned}$$



- Vamos ahora a remplazar el output en la aproximación de Taylor, considerando solo los términos que varían con respecto a O . El cambio de signo cambia el problema de minimizar a maximizar

$$\begin{aligned}
 -\sum_{i=1}^n g * O - \frac{1}{2} * \sum_{i=1}^n h * O - \frac{1}{2} \lambda * O^2 &= -\sum_{i=1}^n g * \frac{-\sum_{i=1}^n g}{\sum_{i=1}^n h + \lambda} - \frac{1}{2} * \left(\sum_{i=1}^n h + \lambda \right) * \left(\frac{-\sum_{i=1}^n g}{\sum_{i=1}^n h + \lambda} \right)^2 \\
 &= \frac{(\sum_{i=1}^n g)^2}{\sum_{i=1}^n h + \lambda} - \frac{1}{2} * \frac{(\sum_{i=1}^n g)^2}{\sum_{i=1}^n h + \lambda} \\
 &= \frac{1}{2} * \frac{(\sum_{i=1}^n g)^2}{\sum_{i=1}^n h + \lambda}
 \end{aligned}$$

XGBoost toma el doble de este valor para simplificar los cálculos

- En el caso de regresión tenemos entonces la medida de similitud de los nodos

$$Sim(nodo) = \frac{(\sum_{i \in nodo} residuo_i)^2}{cardinalidad(hoja) + \lambda}$$

- Se define la cobertura de una hoja como $\sum_{i=1}^n h$. En el caso de regresión, se trata del número de instancias en una hoja. XGBoost define el parámetro de pre poda **min child weight** como la mínima cobertura posible para permitir el particionamiento de un nodo.
 - Se evita el particionamiento cuando los nodos sean muy pequeños.



XGBOOST (CLASIFICACIÓN)

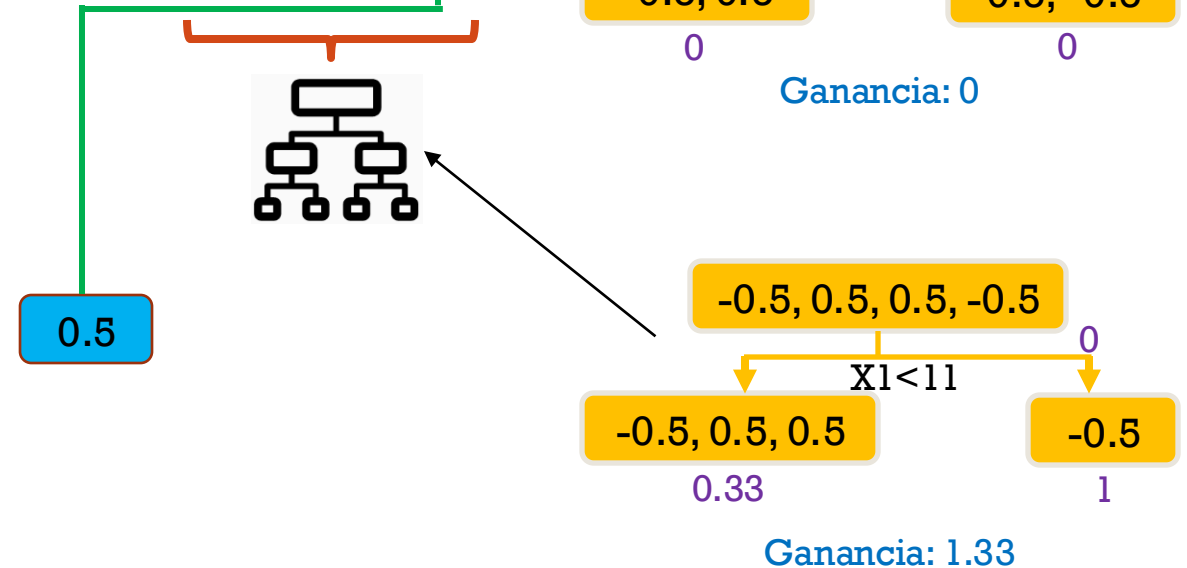
Algoritmo :

1. Se establecen los residuos con respecto a la predicción anterior (valor inicial 0.5)
2. Calcular los pseudo residuos de cada instancia con respecto a la predicción anterior.
3. Para cada nodo evaluar las particiones posibles
4. Calcular la similitud de las instancias de cada

nodo: $Sim(nodo) = \frac{(\sum_{i \in nodo} residuo_i)^2}{\sum_{i \in nodo} p_i * (1 - p_i) + \lambda}$, con p_i la probabilidad anterior del nodo i , y λ que controla la regularización (0 para este ejemplo)

5. Calcular la ganancia de cada partición:
 $Ganancia(\text{árbol}) = Sim(izq) + Sim(der) - Sim(raiz)$
6. Se escoge el particionamiento de mayor ganancia
7. Se itera con los siguientes particionamientos de cada hoja

X1	X2	Y	X1	X2	res
13	..	0	13	..	-0.5
9	..	1	9	..	0.5
7	..	1	7	..	0.5
1	..	0	1	..	-0.5



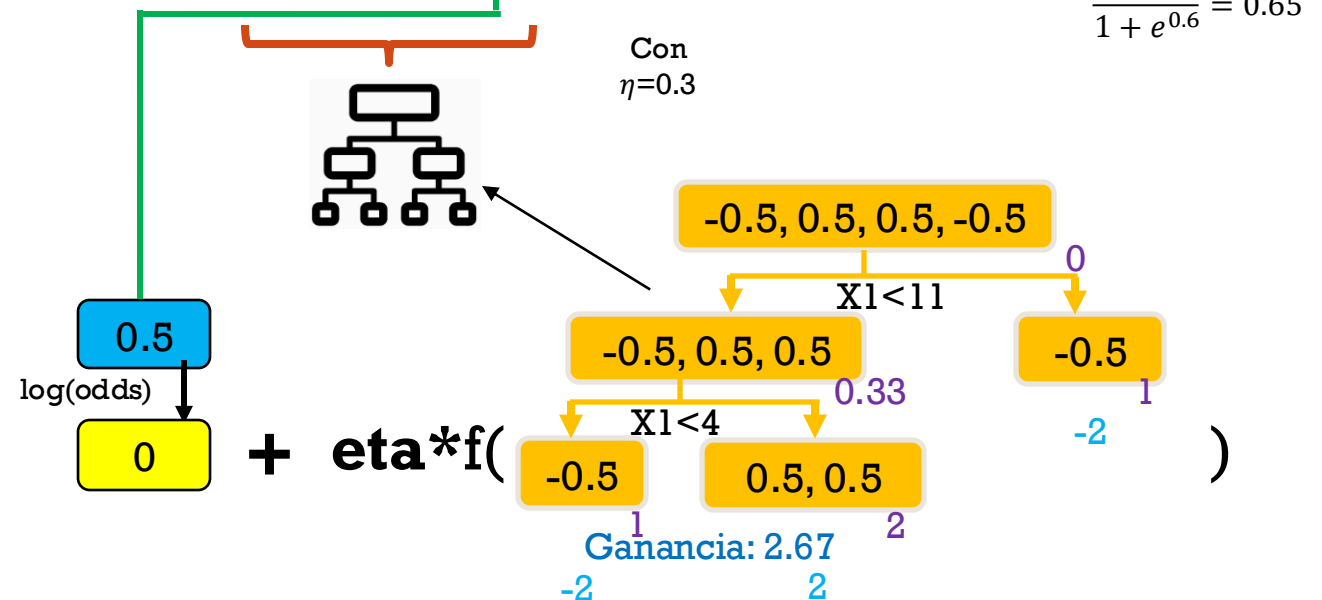
XGBOOST (CLASIFICACIÓN)

Algoritmo :

7. Se itera en los siguientes particionamientos por cada hoja hasta alcanzar un criterio de parada (e.g. **max depth**=2)
8. Una vez desarrollado el árbol, se poda (post-pruning), teniendo en cuenta un umbral mínimo γ (**gamma**) de ganancia para particionar un nodo.
9. Calculamos los valores predichos por el árbol nodo por nodo:

$$Pred(nodo) = \frac{\sum_{i \in nodo} residuo_i}{\sum_{i \in nodo} p_i * (1 - p_i) + \lambda}$$
10. Combinamos el nuevo árbol usando un único **learning rate**, (llamado **eta**, por defecto 0.3). Debemos hacer la combinación sobre los log(odds), y luego transformar los log(odds) a probabilidades
11. Se crean nuevos árboles hasta llegar alcanzar un criterio de parada (e.g. # árboles)

X1	X2	Y	X1	X2	res	PRED
13	..	0	13	..	-0.5	-0.35
9	..	1	9	..	0.5	0.65
7	..	1	7	..	0.5	0.65
1	..	0	1	..	-0.5	-0.35



XGBOOST (CLASIFICACIÓN)

- Para clasificación se usa la función de pérdida

$$L(y_i, F(x_i)) = -(y_i * \log(F(x_i)) + (1 - y_i) * \log(1 - F(x_i)))$$
$$L(y_i, \log(odds)) = -y_i * \log(odds) + \log(1 + e^{\log(odds)})$$

- Su gradiente **g** (primera derivada) es (como vimos en gradient boosting):

$$\mathbf{g} = \frac{\partial L(y_i, \log(odds))}{\partial \log(odds)} = -y_i + \frac{e^{\log(odds)}}{1 + e^{\log(odds)}} = -(y_i - p_i)$$

- Su hessiano **h** (segunda derivada) es:

$$\mathbf{h} = \frac{\partial^2 L(y_i, \log(odds))}{\partial \log(odds)^2} = \frac{e^{\log(odds)}}{1 + e^{\log(odds)}} * \frac{1}{1 + e^{\log(odds)}} = p_i * (1 - p_i)$$



- Para cada hoja buscamos la salida O que minimice $(\sum_{i=1}^n L(y_i, F_{m-1}(x_i) + O)) + \frac{1}{2} \lambda * O^2$, con λ para regularización

Usamos aproximación de Taylor de orden 2, como ya fue ilustrado en el caso de regresión:

$$O = \frac{-\sum_{i=1}^n g}{\sum_{i=1}^n h + \lambda} = \frac{\sum_{i=1}^n (y_i - p_i)}{\sum_{i=1}^n p_i * (1 - p_i) + \lambda} = \frac{\text{suma de los residuos de la hoja}}{\text{suma de proba anterior} * (1 - \text{proba anterior}) + \lambda}$$

- En el caso de clasificación tenemos entonces la medida de similitud de los nodos

$$Sim(nodo) = \frac{(\sum_{i=1}^n g)^2}{\sum_{i=1}^n h + \lambda} = \frac{(\text{suma de los residuos de la hoja})^2}{\text{suma de proba anterior} * (1 - \text{proba anterior}) + \lambda}$$

- Se define la cobertura de una hoja como $\sum_{i=1}^n h$. En el caso de clasificación, la cobertura es $\sum_{i=1}^n p_i * (1 - p_i)$. El parámetro de pre poda **min child weight** se utilizaría para parar el particionamiento de los árboles cuando se llega a un grado de pureza dado.



XGBOOST

Control del overfitting a partir de varios parámetros:

- λ (**lambda**): regularización L_2 con respecto a la similitud y al output de los árboles
 - A mayor λ , menor overfitting porque se reduce la ganancia en cada hoja
- γ (**gamma**): parámetro de post-poda, controla el tamaño de los árboles.
 - A mayor γ , más difícil particionar un nodo. Así γ sea 0, se impiden particionamientos con ganancia negativa.
- **max depth**: parámetro de pre poda, limita el desarrollo de los árboles (6, por defecto).
 - A menor **max depth**, menor overfitting
- **min child weight**: parámetro de pre poda, limita el desarrollo de los árboles. Por defecto es 1, lo cual no afecta modelos de regresión, pero sí los de clasificación.
 - A mayor **min child weight**, menor el overfitting



XGBOOST

Control del overfitting a partir de varios parámetros:

- **η (eta)**: learning rate, controla la velocidad de aprendizaje
 - A menor **η** , menor overfitting, pero menor velocidad de convergencia
- **α (alpha)**: regularización L_1
 - A mayor **α** , menor overfitting
- **subsample**: fracción de la muestra considerada para creación de cada árbol (1 por defecto)
 - A menor valor, menor overfitting (Principio de Bagging)
- **colsample_bytree**: fracción de atributos considerados para creación de las ramas (1 por defecto)
 - A menor valor, menor overfitting (Principio de Random Subspaces)
- **scale_pos_weight**: ayuda a controlar el desbalanceo (1 por defecto), se multiplica al gradiente de la clase minoritaria. Si se exagera en su valor, puede llevar al overfitting



XGBOOST

- Optimización para grandes volúmenes de datos:
 - No se consideran todos los particionamientos, se usan solo los **cuantiles ponderados** (cada dato tiene un peso asociado a la confianza de su predicción – el hessiano **h**)
 - Algoritmo distribuido aproximadamente greedy: divide los datos y calcula una aproximación de los cuantiles de manera **paralela**, para acelerar el entrenamiento
 - Hay 33 cuantiles por defecto (parámetro **sketch_eps**)
- Permite el manejo de **valores faltantes (VFs)**:
 - Se crean las particiones sin incluir los VFs se calculan 2 ganancias, una con los VFs en la rama izquierda, otra con los VFs en la rama derecha
 - Se escoge la partición con mayor ganancia, asociando los VFs a la rama correspondiente



XGBOOST

- Permite hacer **Cross-Validation** en cada iteración
- Permite **retomar el entreno** de un modelo después de haber parado el entrenamiento
- Consideración del **hardware** disponible para el cómputo:
 - Utiliza **memoria cache** para almacenar los gradientes y hessianos buscando optimizar la velocidad del cálculo de similitudes, ganancias y valores predichos
 - Para volúmenes de datos grandes (no caben en RAM) organiza y comprime pedazos del dataset de manera óptima, (**out of core hard drives**).



REFERENCIAS

- *An Introduction to Statistical Learning with Applications in Python. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. Springer. 2023.*
- *Müller, A. C., & Guido, S. (2016). Introduction to machine learning with Python: a guide for data scientists. "O'Reilly Media, Inc".*
- *Diapositivas de clase*
- *Guías de lectura*
- *Cuadernos Jupyter (Intu)*

